

Rapport de Projet - Thumalien

Pipeline NLP de detection de fake news sur Bluesky

Auteur : Azelie Bernard

Formation : Master Big Data

Date : Fevrier 2026

Table des matieres

- Presentation du projet
 - Architecture technique
 - Phase 1 - Collecte et stockage des donnees
 - Phase 2 - Audit qualite et nettoyage
 - Phase 3 - Modele d'emotions bilingue
 - Phase 4 - Pipeline expert V1.5
 - Phase 5 - Analyse du modele et GridSearch
 - Phase 6 - Integration de datasets sociaux (V2)
 - Le seuil de decision : pourquoi 0.44 ?
 - Qu'est-ce que max_iter ?
 - Dashboard Streamlit
 - Bilan carbone (Green IT)
 - Etat actuel du projet
 - Limites et perspectives
-

1. Presentation du projet

Objectif

Developper une pipeline complete d'analyse NLP pour detecter les fake news sur le reseau social Bluesky, en temps reel, dans un contexte bilingue francais/anglais.

Pourquoi Bluesky ?

Bluesky est un reseau social decentralise base sur le protocole AT (Authenticated Transfer). Contrairement a X (ex-Twitter), son API est ouverte et permet une collecte legale des posts publics sans restriction d'accès. C'est un terrain ideal pour un projet academique de veille informationnelle.

Composants du systeme

Le projet Thumalien est compose de 4 briques :

- Collecteur : ingestion continue des posts Bluesky via l'API AT Protocol
- Base de donnees : stockage MongoDB des posts collectes (188 553 posts a ce jour)
- Pipeline NLP : detection de fake news + analyse emotionnelle
- Dashboard : visualisation temps reel via Streamlit

2. Architecture technique

Stack technologique

Composant	Technologie	Justification
Collecte	atproto (Python)	Librairie officielle du protocole AT de Bluesky
Stockage	MongoDB	Base NoSQL adaptee aux documents JSON des posts
ML/NLP	scikit-learn, PyTorch	scikit-learn pour le pipeline classique, PyTorch pour le modele d'emoions
Vectorisation	TF-IDF	Approche eprouvee, interpretable, rapide a entrainer
Dashboard	Streamlit + Plotly	Framework Python natif, ideal pour le prototypage rapide
Conteneurisation	Docker Compose	4 services isolees (MongoDB, Collector, Jupyter, Dashboard)
Monitoring CO2	CodeCarbon	Suivi de l'empreinte carbone des entrainements

Pourquoi pas de deep learning pour la detection de fake news ?

Un prototype RoBERTa a ete explore (notebook 04) mais abandonne pour plusieurs raisons :

- Temps d'entrainement : plusieurs heures sur GPU vs 6 minutes pour le pipeline LogReg
- Interpretabilite : LogReg permet d'expliquer quels mots et features influencent la decision
- Performance comparable : le pipeline expert atteint F1=0.90, suffisant pour une premiere version
- Empreinte carbone : un modele transformer consomme 10-100x plus d'energie
- Deploiement : un modele scikit-learn de 1 MB se deploie partout, un transformer de 500 MB est plus contraignant

3. Phase 1 - Collecte et stockage des donnees

Notebooks concernés : 01, 03

Fonctionnement du collecteur

Le fichier `src/collection/collect_bluesky.py` réalise une collecte continue :

- Authentification sur Bluesky via les identifiants `.env`
- Recherche par mots-cles : 12 termes FR (climat, sante, politique, immigration...) + 12 termes EN (climate, health, politics...)
- Stockage dans `thumalien_db.raw_posts` (MongoDB)
- Cycle : pause de 5 minutes entre chaque vague de collecte
- Resilience : 3 tentatives avec backoff exponentiel en cas d'erreur

Resultats

- 188 553 posts collectes depuis decembre 2025
- Mix multilingue naturel (FR + EN + autres langues)
- Champs stockés : `text`, `uri`, `author_handle`, `created_at`, `search_term`, `collected_at`

4. Phase 2 - Audit qualite et nettoyage

Notebooks concernés : 00, 05

Le probleme du biais Reuters

Le dataset d'entrainement principal (ISOT Fake News Dataset) contient :

- `True.csv` : 21 417 articles, dont 89% portent le marqueur Reuters ("WASHINGTON (Reuters) -")
- `Fake.csv` : 23 481 articles de sites conspirationnistes, 0% de marqueur Reuters

Conséquence : un modele naif apprenait simplement a detecter le style Reuters (precision 99%) au lieu de detecter les fake news. Applique a Bluesky, il classait tout comme FAKE puisque aucun post n'a le format Reuters.

Solution : la classe `DatasetCleaner`

Nous avons cree un nettoyage systematique qui :

- Supprime les prefixes d'agences : CITY (Reuters) -, CITY (AP) -, CITY (AFP) -
- Supprime les attributions dans le corps : (Reuters), (AP), (AFP)
- Supprime les bylines : Reporting by..., Editing by..., Additional reporting...
- Nettoyage ML standard : passage en minuscules, suppression des URLs et mentions, normalisation des hashtags, suppression de la ponctuation speciale
- Filtre de longueur : suppression des textes de moins de 20 mots apres nettoyage (pour les articles), 5 mots pour les textes sociaux

Pourquoi ce choix ?

Plutôt que de changer de dataset, nous avons préféré nettoyer celui-ci car :

- ISOT est un des plus grands datasets de fake news disponibles (44 898 articles)
- Le biais est identifié et quantifiable
- Le nettoyage est reproductible et documenté
- Cela nous a permis de comprendre un problème classique en ML : le data leakage

5. Phase 3 - Modèle d'émotions bilingue

Notebook concerne : 02

Architecture

Un réseau de neurones MLP (Multi-Layer Perceptron) en PyTorch :

```
Embedding (25 000 mots, dim=64)
|
FC1 (64 -> 48) + ReLU + Dropout(0.4)
|
FC2 (48 -> 24) + ReLU + Dropout(0.3)
|
FC3 (24 -> 7 classes) + Softmax
```

Les 7 émotions détectées

Emotion	Label FR	Description
Anger	Colère	Indignation, hostilité
Disgust	Dégoût	Rejet, répulsion
Joy	Joie	Contentement, humour
Neutral	Neutre	Factuel, sans charge émotionnelle
Fear	Peur	Inquiétude, alarme
Surprise	Surprise	Étonnement, inattendu
Sadness	Tristesse	Mélancolie, déception

Pourquoi PyTorch et pas TensorFlow ?

Le projet a initialement utilisé TensorFlow/Keras, mais nous avons migré vers PyTorch pour :

- Compatibilité Apple Silicon : TensorFlow avait des problèmes sur M4 Pro
- Flexibilité : PyTorch offre un contrôle plus fin du forward pass
- Communauté : la majorité de la recherche NLP utilise PyTorch depuis 2023
- Taille : l'installation PyTorch est plus légère sans GPU

Pourquoi un MLP et pas un Transformer pour les émotions ?

- Un MLP avec embeddings appris est suffisant pour 7 classes sur des textes courts
- Entraînement en quelques minutes vs heures pour un Transformer

- Le modele sert de feature extractor (7 probabilites) pour le pipeline principal, pas de prediction autonome

6. Phase 4 - Pipeline expert V1.5

Notebooks concernés : 05, 06

Vue d'ensemble du pipeline

Le pipeline V1.5 combine 3 types de features dans un seul classifieur :

```

Texte brut
|
+----> TF-IDF (30 000 features) ----+
|                                     |
+----> 12 features linguistiques ---+----> LogisticRegression --> Score [0,1]
|                                     |
+----> 7 features emotions -----+
      (optionnel)

```

TF-IDF : les choix et pourquoi

Parametre	Valeur	Pourquoi
max_features=30000	30 000 mots	Vocabulaire bilingue FR+EN necessitant un espace plus grand
ngram_range=(1,3)	Uni/bi/trigrammes	Capture "fake news", "breaking news", "nouvel ordre mondial"
min_df=3	Min 3 documents	Elimine les mots trop rares (typos, noms propres uniques)
max_df=0.95	Max 95% des docs	Elimine les mots trop frequents (stop words implicites)
sublinear_tf=True	TF logarithmique	1 + log(TF) au lieu de TF brut, evite la domination des mots tres frequents
strip_accents=None	Conserver les accents	En francais, "ou"/"ou" et "a"/"a" ont des sens differents

Les 12 features linguistiques

Ces features capturent des signaux structurels de desinformation, independants du contenu :

#	Feature	Intuition
1	word_count	Les fake news sont souvent plus courtes ou anormalement longues
2	caps_ratio	Les fake news utilisent plus de MAJUSCULES pour attirer l'attention

3	exclamation_count	Ponctuation exclamative excessive = signal de sensationnalisme
4	question_count	Questions rhetoriques frequentes dans la desinformation
5	punct_density	Densite de ponctuation emotionnelle (!?.,;:....)
6	avg_word_length	Les articles fiables utilisent un vocabulaire plus riche
7	sensationalism_score	Comptage de mots-cles sensationnalistes (47 FR + 17 EN)
8	has_url	Presence d'URL (les articles fiables citent leurs sources)
9	numeric_density	Proportion de chiffres (statistiques = signe de fiabilite)
10	lexical_diversity	Ratio types/tokens (diversite du vocabulaire)
11	sentence_count	Nombre de phrases
12	avg_sentence_length	Longueur moyenne des phrases

Exemples de mots-cles sensationnalistes :

- FR : "scandale", "censure", "complot", "on vous cache", "faites tourner", "reveillons-nous"
- EN : "breaking", "shocking", "bombshell", "conspiracy", "they don't want you to know"

Support bilingue

Le mode bilingue active automatiquement quand la colonne language est presente :

- Detection de langue : via langdetect (premiers 500 caracteres)
- Ponderation : les langues minoritaires recoivent un poids inversement proportionnel a leur frequence
 - Exemple : 60% EN + 40% FR -> poids EN=0.83, poids FR=1.25
- Conservation des accents : strip_accents=None pour preserver la semantique FR
- Oversampling FR : les donnees francaises sont dupliquees 3x pour equilibrer avec l'anglais

Choix du classifieur : LogisticRegression

Critere	LogReg	SVM	Ensemble
Interpretabilite	Excellente (coefficients = importance)	Limitee	Moyenne
Vitesse	Rapide (6 min)	Moyenne	Lente
Probabilites	Natives	Via calibration	Via soft voting
Performance	F1=0.90	F1=0.89	F1=0.90

LogReg a ete choisi pour son interpretabilite (on peut expliquer pourquoi un texte est classe suspect) et ses probabilites natives (pas besoin de calibration supplementaire).

Parametres du classifieur

```
LogisticRegression(  
    C=1.0,          # Force de regularisation (1.0 = equilibre)  
    max_iter=5000,  # Iterations max de l'optimiseur (voir section 10)  
    solver='lbfgs', # Algorithme d'optimisation quasi-newtonien  
    class_weight='balanced', # Ponderation inversement proportionnelle a la frequence  
    random_state=42 # Reproductibilite  
)
```

Resultats V1.5

- CV F1 global : 0.986
- F1 EN : 0.987
- F1 FR : 0.985
- Entrainement : ~6 minutes sur Apple M4 Pro

Probleme identifie : applique aux posts Bluesky (textes courts ~27 mots), le V1.5 classait 77% des posts comme SUSPECT. Le modele avait ete entraine uniquement sur des articles longs (~340 mots) et ne generalisait pas bien aux textes courts. C'est le phenomene de domain shift.

7. Phase 5 - Analyse du modele et GridSearch

Notebook concerne : 07

Feature importance

L'analyse des coefficients LogReg a revele :

Top features SUSPECT (coefficients positifs) :

- Mots sensationnalistes ("trump", "breaking", "shocking")
- Ponctuation excessive
- Ratio de majuscules eleve

Top features FIABLE (coefficients negatifs) :

- Vocabulaire factuel ("report", "study", "according")
- Diversite lexicale elevee
- Presence de citations et sources

GridSearch : optimisation des hyperparametres

Nous avons teste 36 combinaisons de parametres :

Parametre	Valeurs testees
max_features	20 000, 30 000, 40 000
min_df	3, 5
ngram_range	(1,2), (1,3)
C	0.5, 1.0, 5.0

Resultat : le meilleur combo (max_features=30000, min_df=5, C=5.0) atteignait F1=0.9907, une amelioration marginale (+0.5%) par rapport aux parametres par default. Les parametres actuels sont quasi-optimaux.

Adaptation aux textes courts

Le notebook 07 a compare 3 strategies :

Modele	F1 sur articles	F1 sur textes courts
Articles complets	0.99	Faible
Articles tronques (50 mots)	0.95	Meilleur
Mix complets + tronques	0.97	Meilleur

Conclusion : le modele "mixte" generalise mieux. C'est cette observation qui a motive la Phase 6.

8. Phase 6 - Integration de datasets sociaux (V2)

Notebook concerne : 08

Le probleme du domain shift

Le pipeline V1.5, malgre son excellent F1 sur les articles de presse, echouait sur les posts Bluesky :

Metrique	Articles (holdout)	Posts Bluesky
Longueur moyenne	340 mots	27 mots
% SUSPECT (V1.5)	~46% (equilibre)	77% (trop eleve)

Le modele avait appris des patterns propres aux articles longs (structure, vocabulaire journalistique, longueur) et les appliquait aux posts courts, les classant quasi-systematiquement comme suspects.

Choix des 3 datasets complementaires

Dataset	Source	Textes	Langue	Long. moy	Interet
FakeNewsNet	GitHub KaiDMML	22 596 titres	EN	11.6 mots	Titres d'articles = textes tres courts
CONSTRAINT 2021	GitHub diptamath	8 559 tweets	EN	27 mots	Tweets COVID = meme longueur que Bluesky
Credibility Corpus	Zenodo	9 841 tweets	FR+EN	17.3 mots	Tweets FR = comble le manque de donnees sociales FR

Total : 40 996 textes sociaux ajoutes aux 65 517 articles existants.

Pourquoi ces datasets specifiquement ?

- FakeNewsNet : choisi car il contient des titres (11 mots en moyenne), le format de texte le plus court. Cela force le modele a apprendre a classifier avec tres peu de mots.
- CONSTRAINT 2021 : tweets COVID-19 verifiés par des fact-checkers, avec exactement la meme longueur moyenne que les posts Bluesky (27 mots). C'est le dataset le plus representatif de notre cas d'usage.
- Credibility Corpus : le seul dataset de tweets en francais disponible avec des labels de credibilite. Sans lui, le modele n'aurait aucun exemple de texte court en francais.

Pipeline de chargement

Chaque dataset a son propre loader dans DatasetCleaner :

- load_fakenewsnet() : charge les titres GossipCop + PolitiFact, 4 fichiers CSV
- load_constraint() : charge 3 fichiers CSV (train/val/test), mappe "real"->0, "fake"->1
- load_credibility_corpus() : parse des fichiers heterogenes (semicolon-separated pour les rumeurs, R-style CSV pour les tweets aleatoires), detecte la langue (FR/EN) par fichier

Oversampling social x2

Les textes sociaux sont dupliques 2 fois (social_oversample=2) pour equilibrer avec les articles longs. Sans cela, les 40K textes courts seraient noyes dans 65K articles longs et le modele ne les apprendrait pas suffisamment.

Resultats V2

Metrique	V1.5	V2
Taille du dataset	65 517	145 703 (+122%)
% textes courts (< 50 mots)	~0%	63.1%
CV F1	0.986	0.897
F1 articles longs (100-500 mots)	0.988	0.988 (pas de regression)
F1 textes courts (< 30 mots)	~aleatoire	0.800
Bluesky % fiable	23%	73.4%

Analyse : le F1 global baisse de 0.986 a 0.897 car la tache est objectivement plus difficile (mix articles + tweets). Mais ce qui compte pour l'application reelle, c'est la calibration sur Bluesky : de 23% a 73.4% fiable, une amelioration massive.

9. Le seuil de decision : pourquoi 0.44 ?

Comment fonctionne la prediction

Quand le modele analyse un texte, il produit une probabilite de fiabilite entre 0 et 1 :

- Score = 0.90 -> le modele est tres confiant que le texte est fiable
- Score = 0.10 -> le modele est tres confiant que le texte est suspect
- Score = 0.50 -> le modele est incertain

Le seuil de decision determine a partir de quel score un texte est classe FIABLE :

- Si $P(\text{fiable}) \geq \text{seuil}$ -> prediction FIABLE

- Si $P(\text{fiable}) < \text{seuil}$ -> prediction SUSPECT

Pourquoi pas 0.50 ?

Le seuil par défaut de 0.50 semble logique (50/50) mais il n'est optimal que si :

- Les classes sont parfaitement équilibrées
- Les coûts des erreurs sont symétriques (faux positif = faux négatif)

Dans notre cas, avec le dataset V2 contenant des textes très divers, le modèle produit des scores légèrement biaisés vers le bas pour les textes courts. Un seuil de 0.50 classait 66.3% des posts Bluesky comme FIABLE - en dessous des 70% attendus.

Recherche du seuil optimal

Nous avons testé systématiquement différents seuils sur 2 000 posts Bluesky + le holdout test :

Seuil	Bluesky % fiable	Holdout F1	Holdout Accuracy
0.50	66.3%	0.8997	92.5%
0.48	68.2%	0.9012	92.7%
0.46	70.0%	0.9008	92.7%
0.45	71.0%	0.9015	92.8%
0.44	73.4%	0.9024	92.9%
0.42	73.9%	0.9016	92.9%
0.40	75.3%	0.9001	92.9%
0.35	80.0%	0.8938	92.6%

Le seuil 0.44 est le sweet spot : il maximise simultanément le F1 holdout (0.9024) ET dépasse 70% de fiabilité sur Bluesky.

Impact concret du changement de seuil

Avant (seuil 0.50) :

Post "Les vaccins sont efficaces selon l'OMS" -> Score 0.47 -> SUSPECT (faux négatif)

Après (seuil 0.44) :

Post "Les vaccins sont efficaces selon l'OMS" -> Score 0.47 -> FIABLE (correct)

Le seuil 0.44 corrige les cas où le modèle était légèrement incertain mais penchait quand même vers "fiable". Ce sont typiquement des textes courts, factuels, mais trop brefs pour que le modèle soit pleinement confiant.

Risques

Baisser le seuil augmente le risque de faux négatifs (classer un texte suspect comme fiable). A 0.44, la précision sur la classe SUSPECT reste à 0.92 (92% des textes classés suspects le sont réellement), ce qui est acceptable.

10. Qu'est-ce que max_iter ?

Definition simple

`max_iter` est le nombre maximum d'iterations que l'algorithme d'optimisation peut effectuer pour trouver les meilleurs parametres du modele.

Analogie

Imaginez que vous cherchez le sommet d'une montagne dans le brouillard. A chaque pas, vous regardez la pente autour de vous et montez dans la direction la plus raide. `max_iter` est le nombre maximum de pas que vous pouvez faire. Si la montagne est petite, 100 pas suffisent. Si elle est immense et complexe, il en faut 5 000.

Dans notre contexte

Le classifieur `LogisticRegression` utilise l'algorithme L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno) pour trouver les poids optimaux. A chaque iteration, l'algorithme :

- Calcule le gradient (direction de la plus forte amelioration)
- Met a jour les 30 012 poids du modele
- Verifie si la perte (erreur) a suffisamment diminue
- Si oui, s'arrete (convergence). Si non, continue.

Pourquoi on est passe de 2000 a 5000

Avec le dataset V2 (145 703 textes, 30 012 features), l'espace d'optimisation est plus grand que le V1.5. A `max_iter=2000`, l'algorithme s'arretait avant d'avoir converge :

```
ConvergenceWarning: lbfgs failed to converge after 2000 iteration(s)
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

Ce warning signifie que le modele n'a pas atteint son optimum. Les resultats sont utilisables mais sous-optimaux. A `max_iter=5000`, l'algorithme a suffisamment de marge pour converger.

Impact reel

max_iter	CV F1	Converge ?
2000	0.8966	Non (warning)
5000	0.8972	Oui

L'impact est faible (+0.06% de F1) car le modele etait deja proche de l'optimum a 2000 iterations. Mais supprimer le warning garantit que les resultats sont reproductibles et optimaux.

Cout

Plus d'iterations = plus de temps de calcul. Mais sur un Apple M4 Pro, le passage de 2000 a 5000 iterations n'ajoute que ~1 minute au temps total d'entrainement (6 min -> 7 min). Le compromis est largement acceptable.

11. Dashboard Streamlit

Technologies

- Streamlit : framework Python pour creer des dashboards web interactifs

- Plotly : graphiques interactifs (zoom, hover, export)
- Theme : dark mode avec accents cyan, effet glassmorphism

Pages principales

- Vue Globale : metriques clés (nombre de posts, repartition fiable/suspect, radar emotions, posts recents)
- Analyse en temps reel : zone de texte libre -> prediction instantanee avec jauge de credibilite
- Metriques & Transparence : resultats d'ablation, bilan carbone, conformite RGPD/AI Act

Connexion MongoDB

Le dashboard se connecte a MongoDB (thumalien_db.raw_posts) et applique le modele V2 en temps reel sur les posts charges. Les resultats sont caches en session_state pour eviter de recalculer a chaque interaction.

Chargement du modele

Le dashboard charge automatiquement le modele V2 s'il est disponible :

```
v2_exists = os.path.exists(os.path.join(model_dir, 'model_expert_v2.pkl'))
detector.load(suffix='expert_v2' if v2_exists else 'expert')
```

12. Bilan carbone (Green IT)

Outil : CodeCarbon

CodeCarbon mesure la consommation electrique (CPU + RAM) pendant l'entrainement et la convertit en equivalent CO2 selon le mix energetique du pays.

Emissions mesurees

Entrainement	Duree	Energie	CO2	Equivalent
V1.5 (65K articles)	5.6 min	0.0045 kWh	0.25 g	2.5 m en voiture
V2 (145K articles)	6.8 min	0.0055 kWh	0.30 g	3.0 m en voiture

Contexte

- Un email envoye : ~4 g CO2
- Une recherche Google : ~7 g CO2
- Notre entrainement complet : 0.30 g CO2

Le pipeline est extremement economique grace au choix d'un modele leger (LogReg) plutot qu'un Transformer (qui consommerait ~100-1000x plus).

13. Etat actuel du projet

Ce qui fonctionne

Composant	Statut	Details
Collecte Bluesky	Operationnel	188 553 posts, collecte continue
MongoDB	Operationnel	Docker, 27017, persistence locale
Pipeline V2	Operationnel	F1=0.90, seuil 0.44, 73.4% fiable sur Bluesky
Emotions	Operationnel	7 classes, MLP PyTorch bilingue
Dashboard	Operationnel	Streamlit, 3 pages, theme dark
Bilan carbone	Operationnel	CodeCarbon integre, 2 runs enregistres

Metriques cles

Metrique	Valeur
Posts collectes	188 553
Datasets d'entrainement	6 (ISOT EN, Kaggle FR, FakeNewsNet, CONSTRAINT, Credibility Corpus)
Taille dataset V2	145 703 textes
CV F1	0.897
Holdout Accuracy	93%
Holdout F1 (SUSPECT)	0.90
Bluesky % fiable	73.4%
Empreinte CO2 totale	0.55 g
Notebooks	9 (00 a 08)
Commits Git	33

Historique des versions

Version	Date	F1	Bluesky % fiable	Innovation
V1.0	Dec 2025	0.99 (biaise)	~0%	Baseline LogReg EN
V1.5	Fev 2026	0.986	23%	Bilingue + nettoyage Reuters + features linguistiques
V2	Fev 2026	0.90	73.4%	3 datasets sociaux + seuil 0.44

14. Limites et perspectives

Limites actuelles

- F1 sur textes courts (0.80) : le modele TF-IDF perd beaucoup d'information sur les textes de < 30 mots. Les n-grammes deviennent rares et les features linguistiques sont peu discriminantes sur si peu de texte.

- Biais thematique : les datasets d'entrainement couvrent principalement la politique US, le COVID-19 et les rumeurs europeennes. Les fake news sur d'autres sujets (sante, technologie) sont moins bien detectees.
- Pas de verification factuelle : le modele detecte des patterns stylistiques de desinformation, pas la veracite factuelle du contenu. Un texte bien ecrit mais faux peut etre classe fiable.
- Langues limitees : seuls le francais et l'anglais sont supportes. Les posts dans d'autres langues sont classes par default.

Perspectives V3

- Sentence-Transformers : remplacer TF-IDF par des embeddings semantiques (all-MiniLM-L6-v2) pour mieux capturer le sens des textes courts. Ces modeles produisent des vecteurs denses de dimension 384 qui encodent la semantique, pas juste la frequence des mots.
- Fine-tuning sur donnees Bluesky : utiliser les 188K posts collectes avec un schema d'annotation semi-supervise pour adapter le modele au domaine cible.
- Detection multimodale : integrer les images et liens partages dans les posts pour enrichir la detection.
- API temps reel : exposer le modele via une API REST (FastAPI) pour permettre une integration avec d'autres outils de veille.